```
    do_something_else();
});
```

Once you've started your thread, you need to explicitly decide whether to wait for it to finish (by joining with it—see section 2.1.2) or leave it to run on its own (by detaching it—see section 2.1.3). If you don't decide before the `std::thread` object is destroyed, then your program is terminated (the `std::thread` destructor calls `std::terminate()`). It's therefore imperative that you ensure that the thread is correctly joined or detached, even in the presence of exceptions. See section 2.1.3 for a technique to handle this scenario. Note that you only have to make this decision before the `std::thread` object is destroyed—the thread itself may well have finished long before you join with it or detach it, and if you detach it, then the thread may continue running long after the `std::thread` object is destroyed.

If you don't wait for your thread to finish, then you need to ensure that the data accessed by the thread is valid until the thread has finished with it. This isn't a new problem—even in single-threaded code it is undefined behavior to access an object after it's been destroyed—but the use of threads provides an additional opportunity to encounter such lifetime issues.

One situation in which you can encounter such problems is when the thread function holds pointers or references to local variables and the thread hasn't finished when the function exits. The following listing shows an example of just such a scenario.

Listing 2.1. A function that returns while a thread still has access to local variables